

SHETH MVLU COLLEGE
SUBJECT:-Cyber & Information Security
Practical 2

Aim:

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI

INPUT:-

```
from math import gcd

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d

# Input primes
p = int(input("Enter prime p: "))
q = int(input("Enter prime q: "))

n = p * q
phi = (p - 1) * (q - 1)

e = 17

while gcd(e, phi) != 1:
    e += 2

d = mod_inverse(e, phi)

print("\nPublic Key:", (e, n))
print("Private Key:", (d, n))

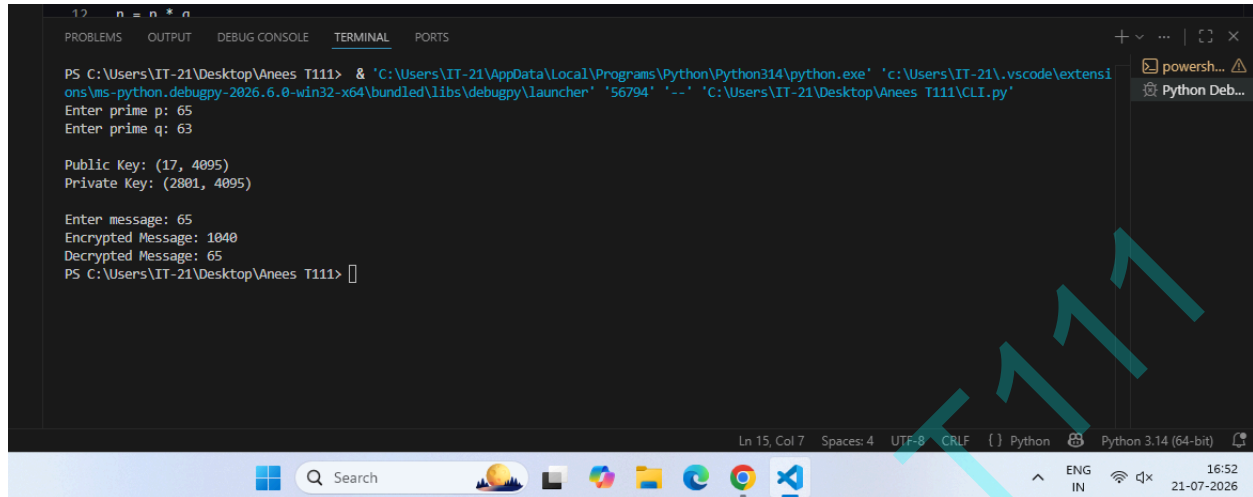
message = int(input("\nEnter message: "))

cipher = pow(message, e, n)
print("Encrypted Message:", cipher)

decrypted = pow(cipher, d, n)
print("Decrypted Message:", decrypted)
```

SHETH MVLU COLLEGE
SUBJECT:-Cyber & Information Security

OUTPUT:-



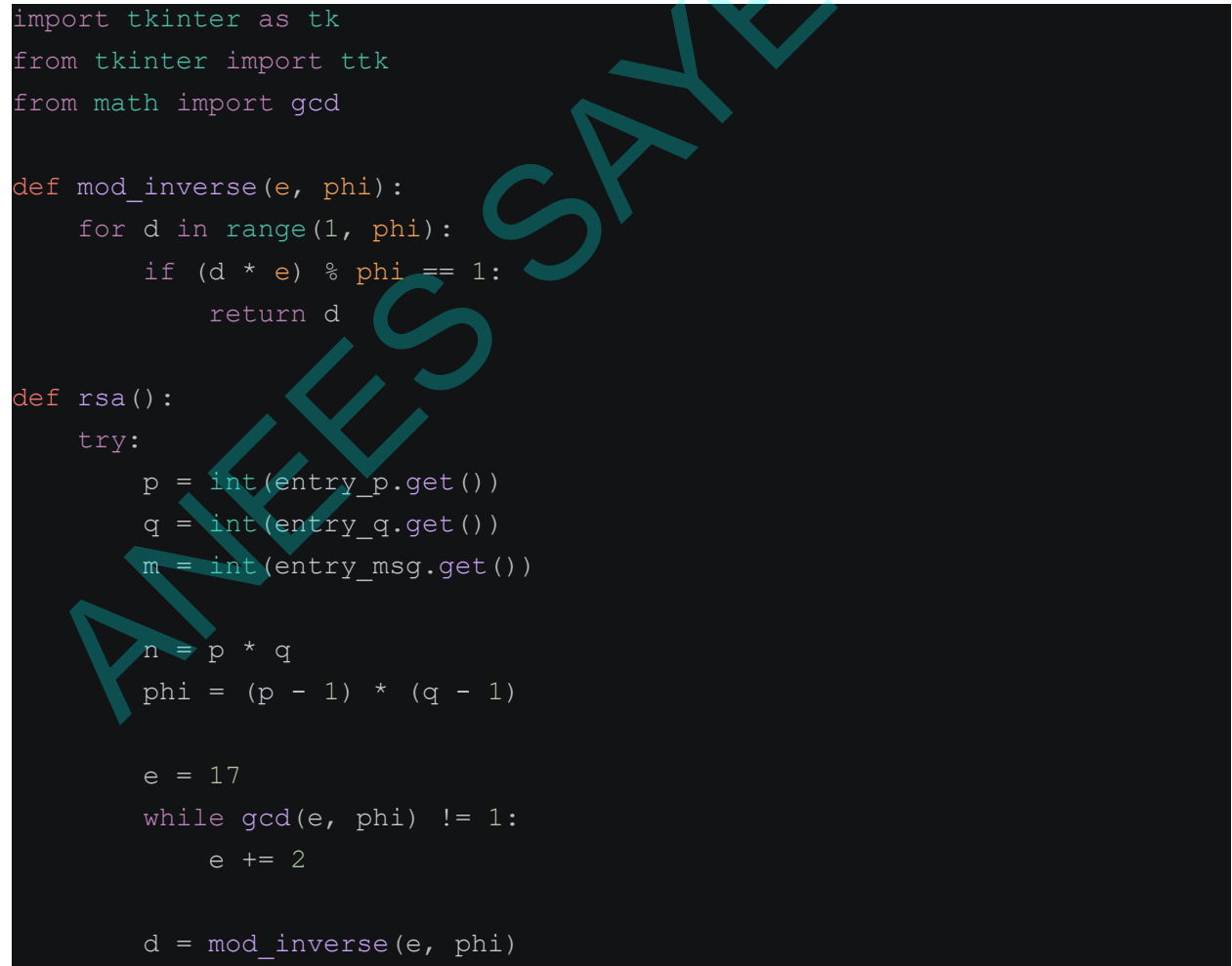
```
PS C:\Users\IT-21\Desktop\Anees T111> & 'C:\Users\IT-21\AppData\Local\Programs\Python\Python314\python.exe' 'C:\Users\IT-21\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '56794' '--' 'C:\Users\IT-21\Desktop\Anees T111\CLI.py'
Enter prime p: 65
Enter prime q: 63

Public Key: (17, 4095)
Private Key: (2801, 4095)

Enter message: 65
Encrypted Message: 1040
Decrypted Message: 65
PS C:\Users\IT-21\Desktop\Anees T111>
```

GUI

INPUT:-



```
import tkinter as tk
from tkinter import ttk
from math import gcd

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d

def rsa():
    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        m = int(entry_msg.get())

        n = p * q
        phi = (p - 1) * (q - 1)

        e = 17
        while gcd(e, phi) != 1:
            e += 2

        d = mod_inverse(e, phi)
```

ANEES SAYED T111

SHETH MVLU COLLEGE
SUBJECT:-Cyber & Information Security

```
        cipher = pow(m, e, n)
        plain = pow(cipher, d, n)

        result.config(
            text=f"Public Key : ({e}, {n})\n"
                f"Private Key : ({d}, {n})\n\n"
                f"Encrypted Message : {cipher}\n"
                f"Decrypted Message : {plain}"
        )

    except:
        result.config(text="Please enter valid numeric values.")

# Main Window
root = tk.Tk()
root.title("RSA Encryption & Decryption System")
root.geometry("550x450")
root.configure(bg="#f4f6f9")
root.resizable(False, False)

# Heading
title = tk.Label(
    root,
    text="RSA Public Key Cryptography",
    font=("Segoe UI", 18, "bold"),
    bg="#f4f6f9",
    fg="#1f4e79"
)
title.pack(pady=15)

# Frame
frame = tk.Frame(root, bg="white", bd=2, relief="groove")
frame.pack(padx=20, pady=10, fill="both")

tk.Label(frame, text="Prime Number (p)",
        font=("Segoe UI", 10),
        bg="white").pack(pady=(15, 5))
entry_p = ttk.Entry(frame, width=30)
entry_p.pack()
```

SHETH MVLU COLLEGE
SUBJECT:-Cyber & Information Security

```
tk.Label(frame, text="Prime Number (q)",
         font=("Segoe UI", 10),
         bg="white").pack(pady=(10, 5))
entry_q = ttk.Entry(frame, width=30)
entry_q.pack()

tk.Label(frame, text="Message",
         font=("Segoe UI", 10),
         bg="white").pack(pady=(10, 5))
entry_msg = ttk.Entry(frame, width=30)
entry_msg.pack()

ttk.Button(
    frame,
    text="Encrypt & Decrypt",
    command=rsa
).pack(pady=20)

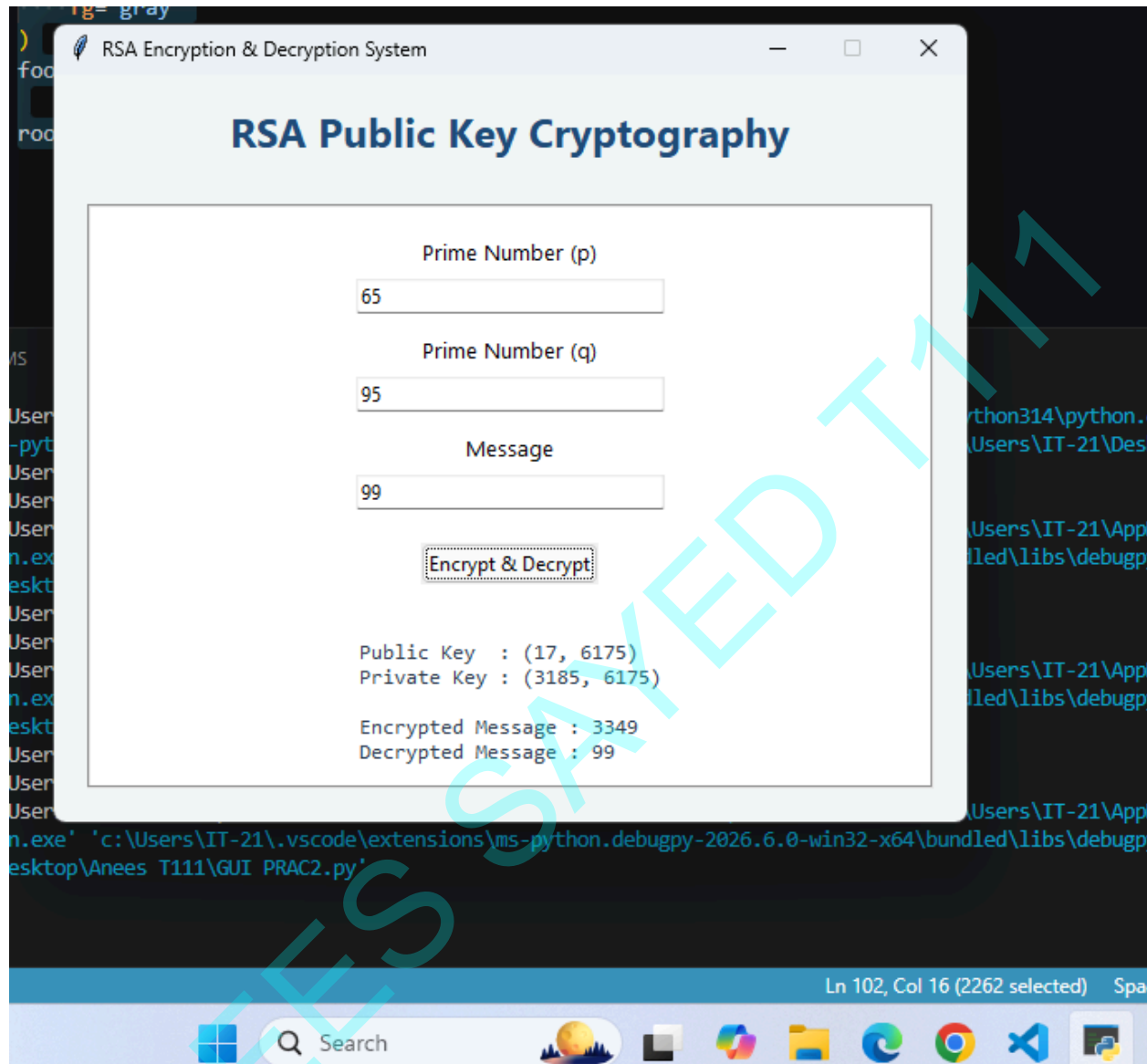
result = tk.Label(
    frame,
    text="",
    font=("Consolas", 10),
    bg="white",
    justify="left",
    fg="#2e4053"
)
result.pack(pady=10)

footer = tk.Label(
    root,
    text="RSA Algorithm Demonstration",
    font=("Segoe UI", 9),
    bg="#f4f6f9",
    fg="gray"
)
footer.pack(side="bottom", pady=10)

root.mainloop()
```

SHETH MVLU COLLEGE
SUBJECT:-Cyber & Information Security

OUTPUT:-



ANEESSAYED T111